# 第2章 算法入门

# 本章简介

- 伪代码中的约定
- 循环不变式
- 算法分析技术(RAM模型)
- 最坏情况和平均情况分析
- 算法设计的方法(分治法)
- 分治法分析

# 伪代码中的约定

```
INSERTION-SORT(A)
1 for j ← 2 to length[A]
2     do key ← A[j]
3        ▷ Insert A[j] into the sorted sequence A[1 .. j - 1].
4        i ← j – 1
5        while i > 0 and A[i] > key
6            do A[i + 1] ← A[i]
7               i ← i – 1
8        A[i + 1] ← key
```

# 伪代码中的约定

1. 书写上的“缩进”表示程序中的分程序(程序块)结构。
2. while,for,repeat等循环结构和if,then,else条件结构与Pascal中相同，在退出循环后，循环计数器的值仍然保持。
3. “▹”表示后面部分是个注释。
4. 多重赋值i←j←e是将表达式e的值赋给变量i和j;
5. 变量(如i，j和key等)是局部于给定过程的。在没有显式说明的情况下，我们不使用全局变量。

## 伪代码中的约定（续）

6. 数组元素是通过“数组名[下标]”这样的形式来访问的。符号“..”用来表示数组中的一个取值范围，例如: A[1..j]就表示A的一个子数组，它包含了j个元素A[1]，A[2]，…，A[j]。
7. 复合数据一般组织成对象，它们是由属性或域所组成的。域的访问是由域名后跟由方括号括住的对象名形式来表示。例如: 数组可以被看作是一个对象，其属性有length，表示数组中元素的个数，如length[A]就表示数组A中的元素个数。

   用于表示一个对象的变量被看做是指向该对象的一个指针！

## 伪代码中的约定（续）

8. 参数采用按值传递方式，当对象被传递时，实际传递的是一个指向该对象数据的指针，而对象的各个域则不被拷贝。例如，如果x是某个被调用过程的参数，在被调过程中的赋值x←y对主调过程来说是不可见的。但是，赋值f[x]←3却是可见的。
9. 布尔运算符“and”和“or”都具有短路能力。

$$x \neq NIL \quad and \quad f[x] = y$$

不需要当我们试图在$x$为$NIL$时计算$f[x]$

# 2.1插入排序

- 排序问题
  - 输入：n个数〈$a_1, a_2, \ldots, a_n$〉
  - 输出：输入序列的一个排列(重新排序)〈$a'_1, a'_2, \ldots, a'_n$〉,使得$a'_1 \le a'_2 \le \ldots \le a'_n$
- 插入排序算法思想：
  - 与打牌一样
  - 摸起一张牌
  - 将它插入到左手一把牌的正确位置



# 2.1插入排序

- 插入排序伪代码：

```
INSERTION-SORT(A)
1 for j ← 2 to length[A]
2     do key ← A[j]
3         ▷ Insert A[j] into the sorted sequence A[1 .. j - 1].
4         i ← j – 1
5         while i > 0 and A[i] > key
6             do A[i + 1] ← A[i]
7                 i ← i – 1
8         A[i + 1] ← key
```

# Example of insertion sort

8 2 4 9 3 6

```
INSERTION-SORT(A)
1 for j ← 2 to length[A]
2     do key ← A[j]
3        ▷ Insert A[j] into the sorted sequence A[1 · · j - 1].
4        i ← j – 1
5        while i > 0 and A[i] > key
6           do A[i + 1] ← A[i]
7              i ← i – 1
8        A[i + 1] ← key
```

# Example of insertion sort

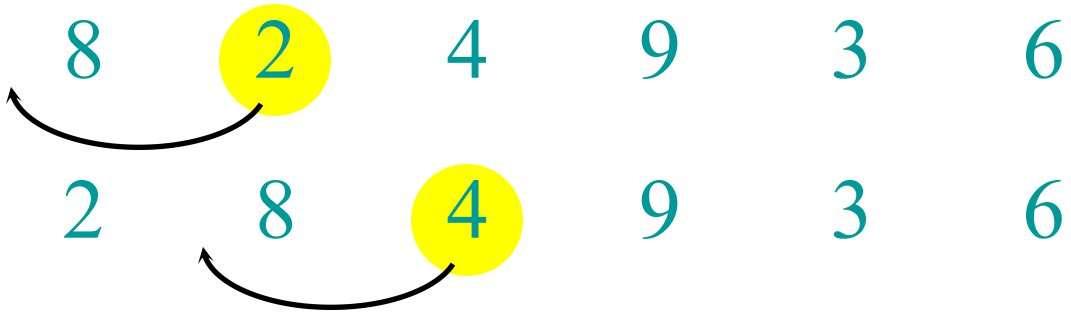


# Example of insertion sort

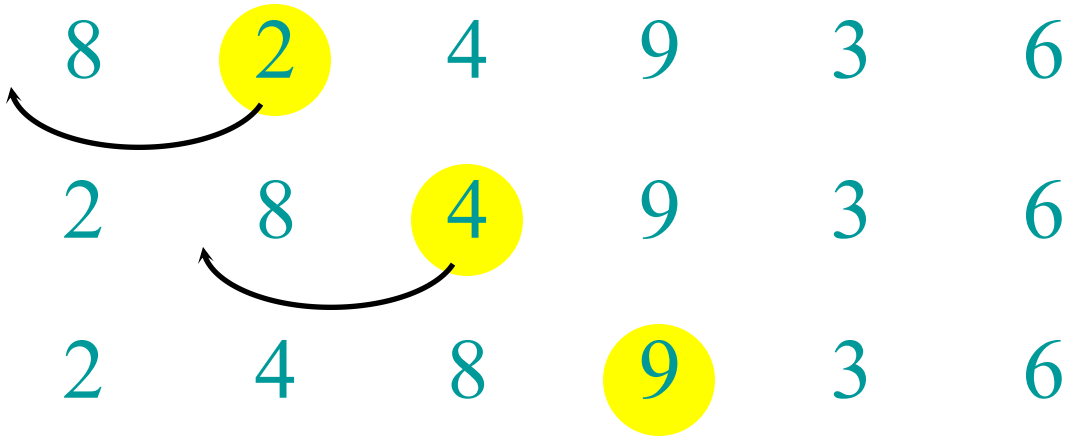



```
INSERTION-SORT(A)
1 for j ← 2 to length[A]
2     do key ← A[j]
3        ▷ Insert A[j] into the sorted sequence A[1 · · j - 1].
4        i ← j – 1
5        while i > 0 and A[i] > key
6           do A[i + 1] ← A[i]
7              i ← i – 1
8        A[i + 1] ← key
```

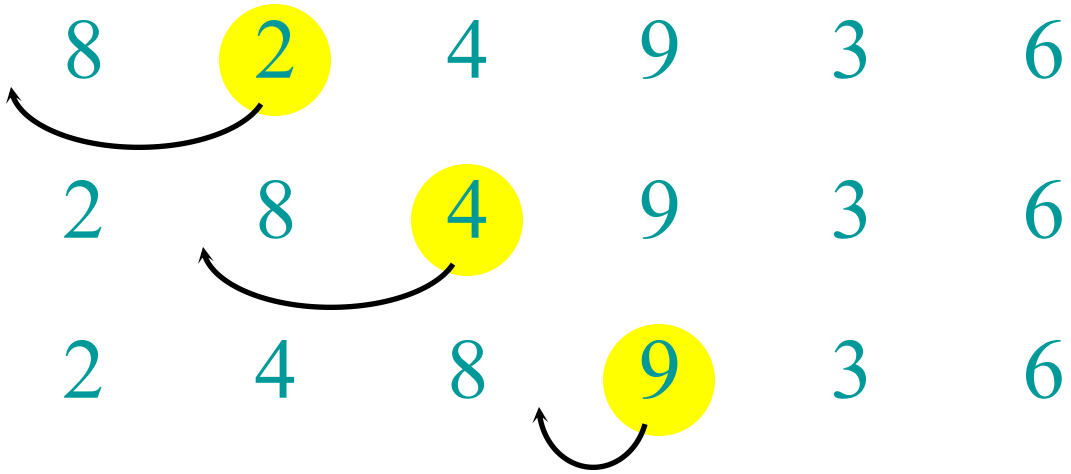
# Example of insertion sort



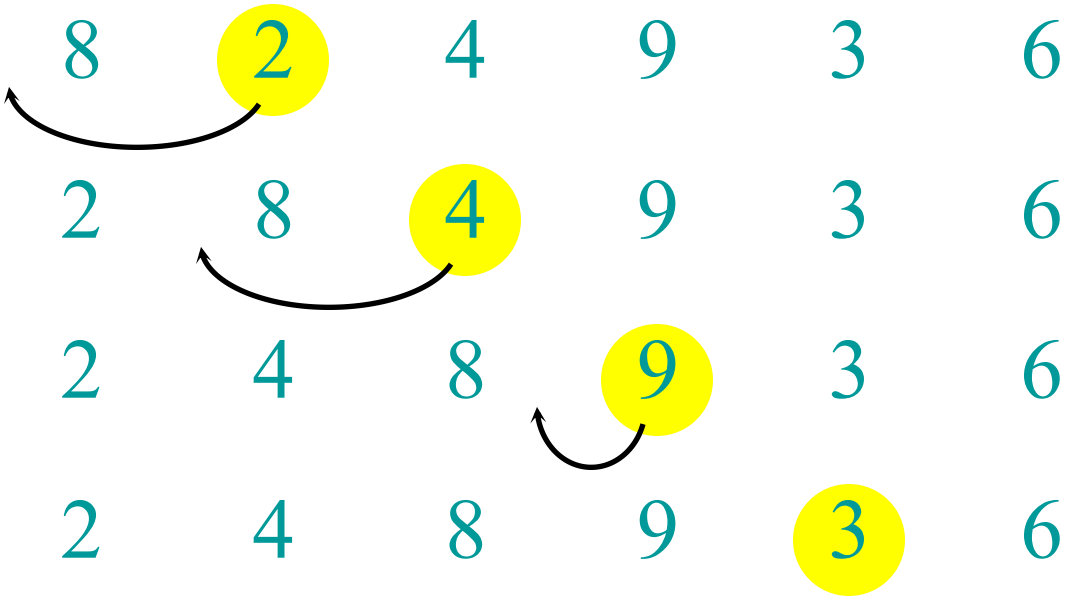
# Example of insertion sort



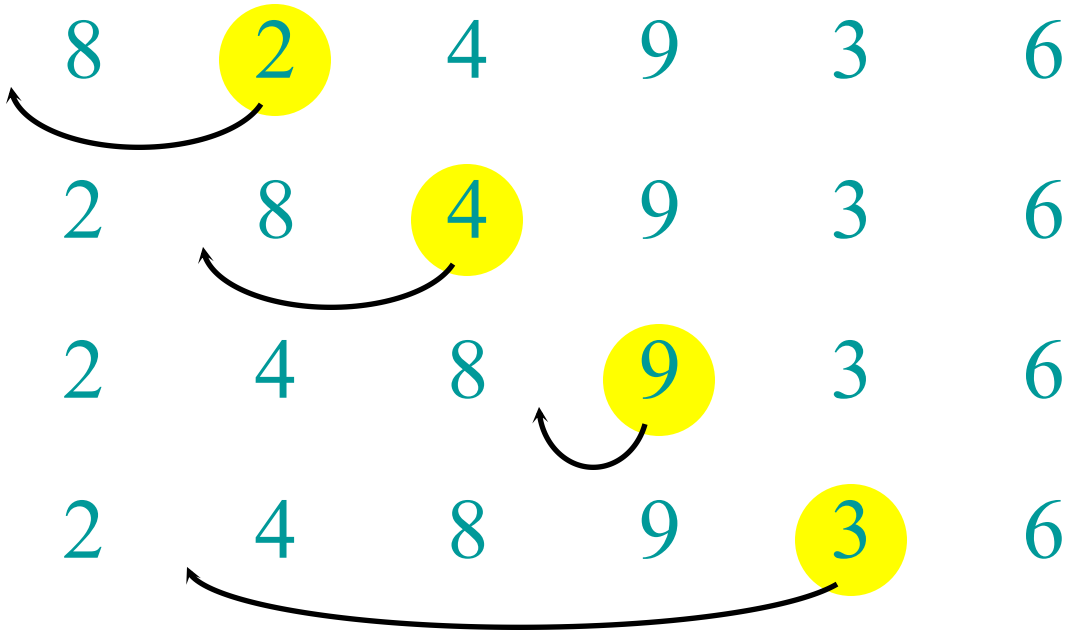
# Example of insertion sort



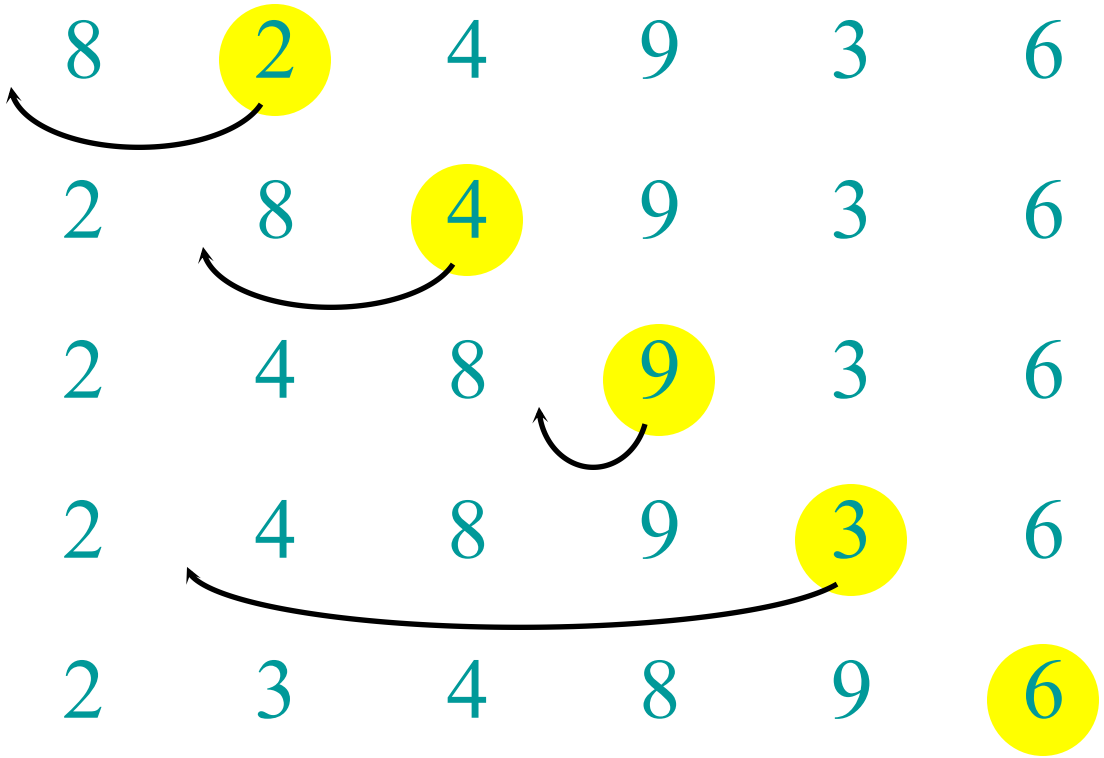
# Example of insertion sort



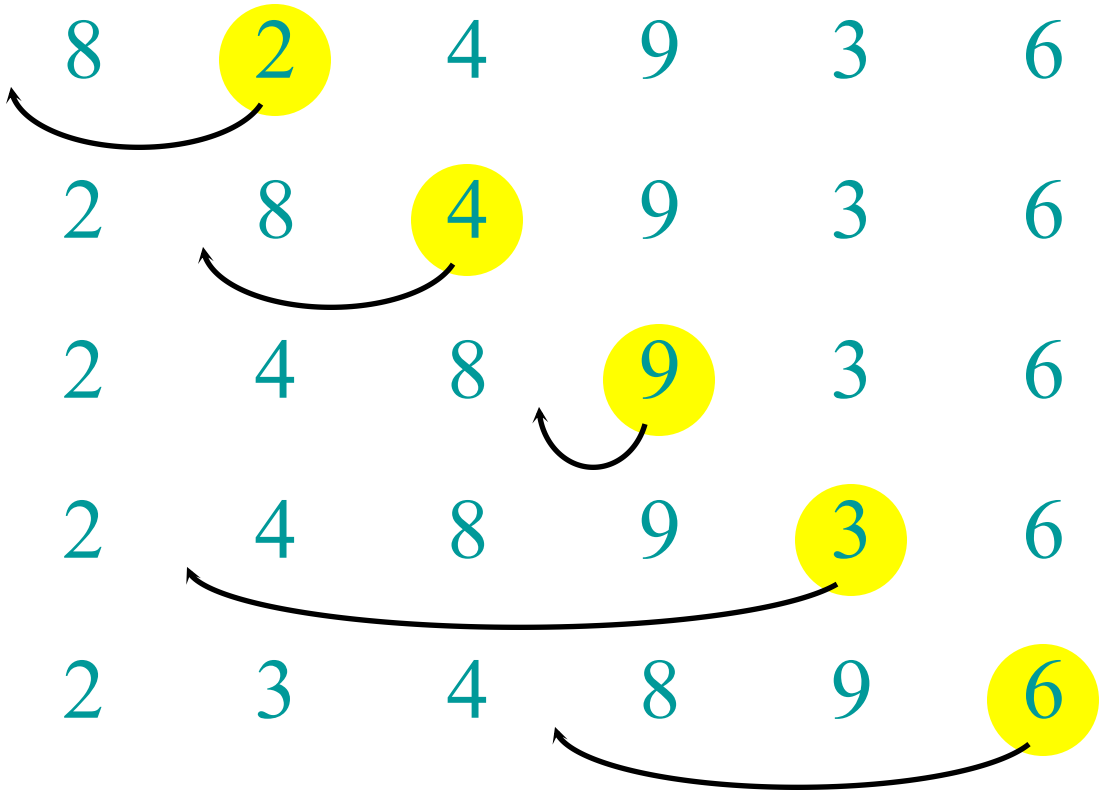
# Example of insertion sort



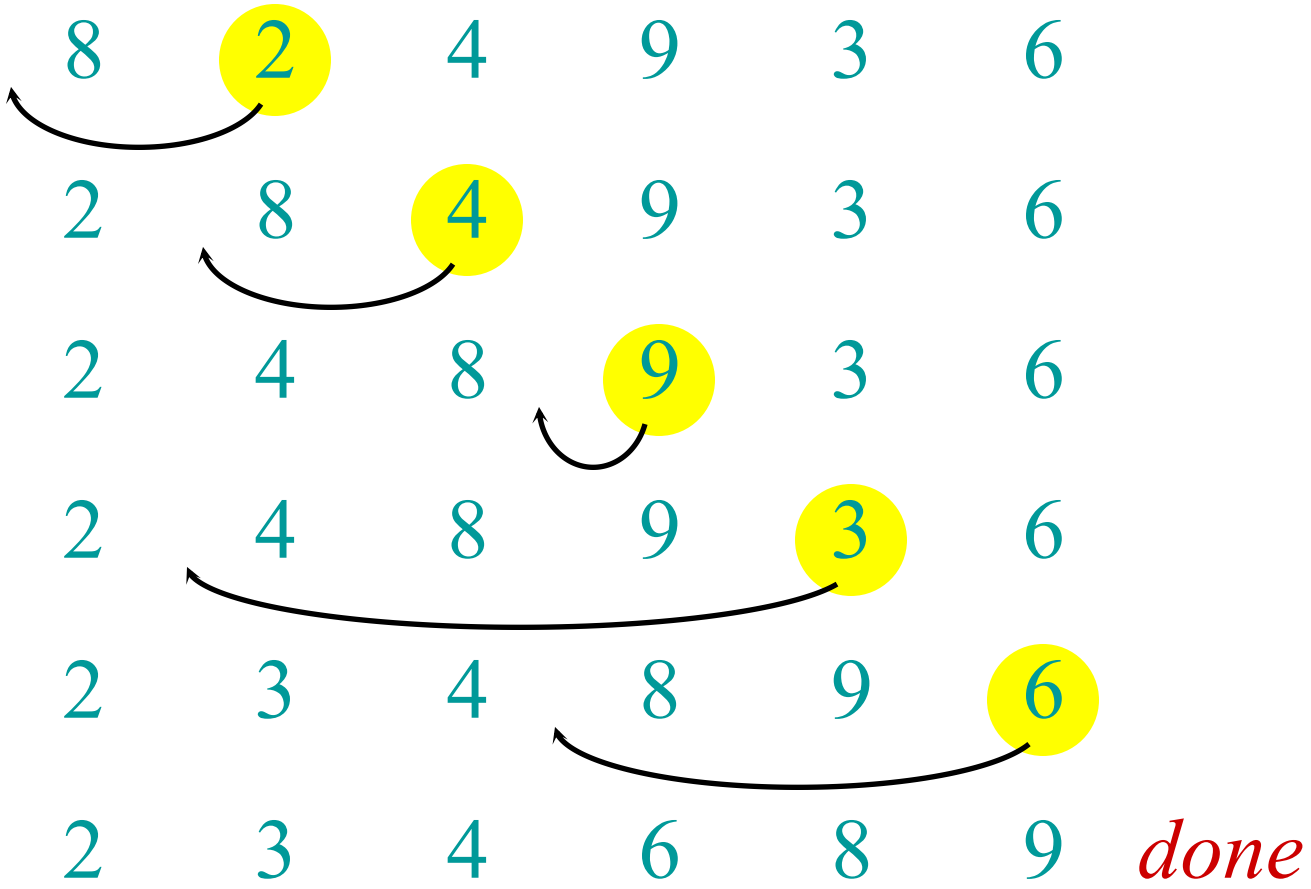
# Example of insertion sort



# Example of insertion sort



# Example of insertion sort



# 2.1循环不变式

- 在循环内及循环结束的时候都成立的表达式
- 一般而言,用这个式子表示希望得到的结果,如果在循环的每一步,这个式子都是正确的,那么循环结束后,这个式子也正确,并得到了期望的结果
- 用于证明算法正确性
- 循环不变式
  - 初始化：它在循环的第一轮迭代开始之前，应该是正确的
  - 保持：如果在循环的某一次迭代开始之前它是正确的，那么，在下一次迭代开始之前，它也应该保持正确。
  - 终止：当循环结束时，不变式给了我们一个有用的性质，它有助于表明算法是正确的。
- 类比数学归纳法

# 2.1插入排序

```
INSERTION-SORT(A)
1 for j ← 2 to length[A]
2    do key ← A[j]
3       ▷ Insert A[j] into the sorted sequence A[1 ‥ j - 1].
4       i ← j – 1
5       while i > 0 and A[i] > key
6          do A[i + 1] ← A[i]
7             i ← i – 1
8       A[i + 1] ← key
```

- 插入排序算法正确性
  - 在外层for循环(循环变量为j)的每一轮迭代开始，包含元素A[1..j-1]的子数组已排好序
    - 初始化：j = 2, 子数组只包含一个元素A[1]，显然有序
    - 保持：在外层for循环的循环体中，要将A[j-1]、A[j-2]、A[j-3]等元素向右移一个位置，直到找到A[j]的适当位置，这时将A[j]的值插入。此时A[1..j]也是有序的
    - 终止：当j大于n时(j＝n+1)，外层for循环结束，将j替换为n+1，子数组为A[1..n]，根据保持性A[1..n]有序，即所需的输出。

# 2.2算法分析

- 算法分析即指对一个算法所需要的资源进行预测。
- 内存、通信带宽或计算机硬件等资源偶尔会是我们主要关心的，但通常，资源是指我们希望测度的计算时间。

# RAM模型

- 分析一个算法之前，需要建立一个实现技术的模型，包括描述所用资源的及代价的模型
- RAM模型：通用的单处理器、随机存取器RAM计算模型
  - 指令一条接一条地执行，没有并发操作。
  - 包含了真实计算机中常见的指令：算术指令(加法、减法、乘法、除法、取余、向下取整、向上取整指令)、数据移动指令(装入、存储、复制指令)和控制指令(条件和非条件转移、子程序调用和返回指令)。
  - 每条指令所需的时间都为常量。
  - 数据类型有整数类型和浮点实数类型

# 2.2算法分析

- RAM模型中的灰色区域：
  - 真实计算机包含的一些以上未列出的指令
  - 例如：指数运算一般情况下不是常量时间的指令，然而，在受限的情况下，指数运算是一种常量时间的操作(“左移”指令)
- RAM没有对常见的存储器层次进行建模
  - 有几种计算模型试图反映出存储层次的效果
  - 有时在真实计算机上的真实程序中非常重要
  - 这些计算模型复杂，比较难以运用
  - RAM模型分析通常能够很好地预测实际计算机上的性能。

# 2.2算法分析

- 插入排序算法的分析
  - 时间开销与输入有关
    - 排序1000个数的时间比排序三个数的时间要长。
    - 排序两个相同长度的输入序列，所需的时间也可能不同
  - 一般来说，算法所需时间是与输入规模同步增长的，因而一个程序的运行时间是其输入的函数。
  - 输入规模
    - 最自然的度量标准是输入中的元素个数
  - 算法的运行时间
    - 独立于具体机器的“步骤”概念。
    - 假定每次执行第i行所花的时间都是常量c_i

# 2.2算法分析

## 插入排序算法伪码

| INSERTION-SORT($A$) | *cost* | *times* |
|---|---|---|
| 1 **for** $j \leftarrow 2$ **to** *length*[$A$] | $c_1$ | $n$ |
| 2 **do** $key \leftarrow A[j]$ | $c_2$ | $n$- 1 |
| 3 ▷ Insert $A[j]$ into the sorted sequence $A[1 \,..\, j - 1]$. | 0 | $n-1$ |
| 4 $i \leftarrow j$ -1 | $c_4$ | $n-1$ |
| 5 **while** $i > 0$ and $A[i] >$ key | $c_5$ | $\sum_{j=2}^{n} t_j$ |
| 6 **do** $A[i + 1] \leftarrow A[i]$ | $c_6$ | $\sum_{j=2}^{n} (t_j - 1)$ |
| 7 $i \leftarrow i$ -1 | $c_7$ | $\sum_{j=2}^{n} (t_j - 1)$ |
| 8 $A[i + 1] \leftarrow key$ | $c_8$ | $n$ -1 |

# 2.2算法分析

## 插入排序算法总运行时间

$$T(n)=c_1n+c_2(n-1)+c_4(n-1)+c_5\sum_{j=2}^{n}t_j+c_6\sum_{j=2}^{n}(t_j-1)+c_7\sum_{j=2}^{n}(t_j-1)+c_8(n-1)$$

- 如果已经排好序

$$t_j=1$$

$$T(n)=c_1n+c_2(n-1)+c_4(n-1)+c_5(n-1)+c_8(n-1)$$

$$=(c_1+c_2+c_4+c_5+c_8)n-(c_2+c_4+c_5+c_8)$$

- 如果逆序

$$t_j=j$$

$$\sum_{j=2}^{n}j=\frac{n(n-1)}{2}-1;\quad\sum_{j=2}^{n}(j-1)=\frac{n(n-1)}{2}$$

$$T(n)=c_1n+c_2(n-1)+c_4(n-1)+c_5\left(\frac{n(n-1)}{2}-1\right)+c_6\left(\frac{n(n-1)}{2}\right)+c_7\left(\frac{n(n-1)}{2}-1\right)+c_8(n-1)$$

$$=\left(\frac{c_5}{2}+\frac{c_6}{2}+\frac{c_7}{2}\right)n^2+\left(c_1+c_2+c_4+\frac{c_5}{2}-\frac{c_6}{2}-\frac{c_7}{2}+c_8\right)n-(c_2+c_4+c_5+c_8)$$

# 2.2算法分析

- 最坏情况和平均情况分析
  - 对于插入排序
    - 最好情况：排好序
    - 最坏情况：逆序
  - 一般考察算法的最坏情况运行时间
    - 一个算法的最坏情况运行时间是在任何输入下运行时间的一个上界。
    - 对于某些算法来说，最坏情况出现得相当频繁的。例如，当在数据库中检索一条信息时，要检索的信息常常是数据库中没有的。
    - 大致上看来，“平均情况”通常与最坏情况一样差。(插入排序中平均情况下，*A[1..j-1]*中的一半元素小于*A[j]*)

# 2.2算法分析

- 最坏情况和平均情况分析(续)
  - 在某些特定的情况下，我们可能会对一个算法的平均情况或期望的运行时间感兴趣
  - 第5章将介绍概率分析(probabilistic analysis)技术，它可以用来确定一个算法期望的运行时间。

# 2.2算法分析

- 增长的量级
  - 忽略每条语句的真实代价，而用常量$c_i$来表示
  - 更加简单，忽略常系数（多项式系数）对$c_i$依赖性
  - 只考虑公式中的最高次项
    - 低阶项相当来说不太重要。
  - 忽略最高次项的常数系数
    - 相对于增长率来说，系数是次要的。
- 如果一个算法的最坏情况运行时间要比另一个算法的低，我们就常常认为它的效率更高。
- 在输入的规模较小时，由于常数项和低次项的影响，这种看法有时可能是不对的
- 对规模足够大的输入来说，这种看法总是对的

# 2.3算法设计方法—分治法

- 插入排序使用的是增量（incremental）方法
  - 在排好子数组A[1..j-1]后，将元素A[j]插入，形成排好序的子数组A[1..j]。
- 另一种设计策略：分治法（Divide and Conquer）
  - 结构上是递归的
  - 将原问题划分成n个规模较小而结构与原问题相似的子问题；递归地解决这些子问题，然后再合并其结果，就得到原问题的解。
  - 优点之一是可以利用在第4章（递归式）中介绍的技术，很容易地确定其运行时间。

# 2.3.1分治法

- 分治模式在每一层递归上都有三个步骤：
  - 分解（Divide）：将原问题分解成一系列子问题；
  - 解决（Conquer）：递归地解各子问题。若子问题足够小，则直接求解；
  - 合并（Combine）：将子问题的结果合并成原问题的解。
- 合并排序（merge sort）算法完全依照了上述模式，直观地操作如下：
  - 分解：将n个元素分成各含n/2个元素的子序列；
  - 解决：用合并排序法对两个子序列递归地排序；
  - 合并：合并两个已排序的子序列以得到排序结果。
- 在对子序列排序时，其长度为1时递归结束。单个元素被视为是已排好序的。

# 2.3.1分治法

- 总结可得MERGE-SORT伪代码

```
MERGE-SORT(A, p, r)
1 if p < r
2     then q ← ⌊(p + r)/2⌋
3       MERGE-SORT(A, p, q)
4       MERGE-SORT(A, q + 1, r)
5       MERGE(A, p, q, r)
```

# 2.3.1分治法

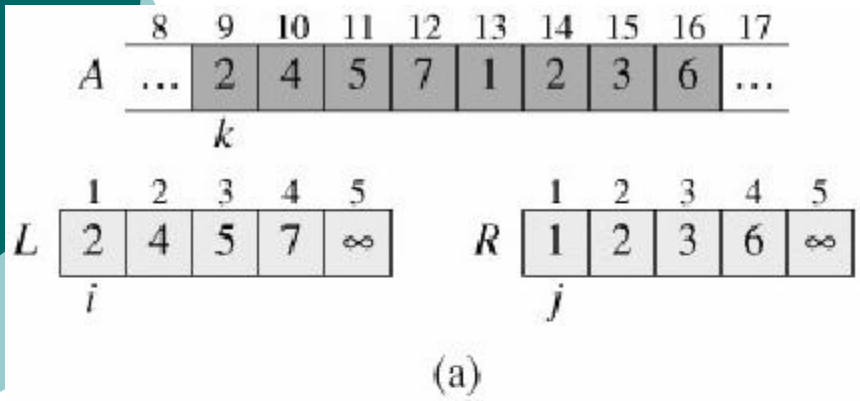
- 合并排序的关键步骤在于合并步骤中的合并两个已排序子序列
- 引入一个辅助过程MERGE(A，p，q，r)，其中A是个数组，p、q和r是下标，满足$p \le q < r$。A[p..q]和A[q+1..r]都已排好序
- MERGE过程的时间代价为$\Theta(n)$,n=r-p+1
- 利用∞来作为哨兵值，用于简化代码。

# 2.3.1分治法

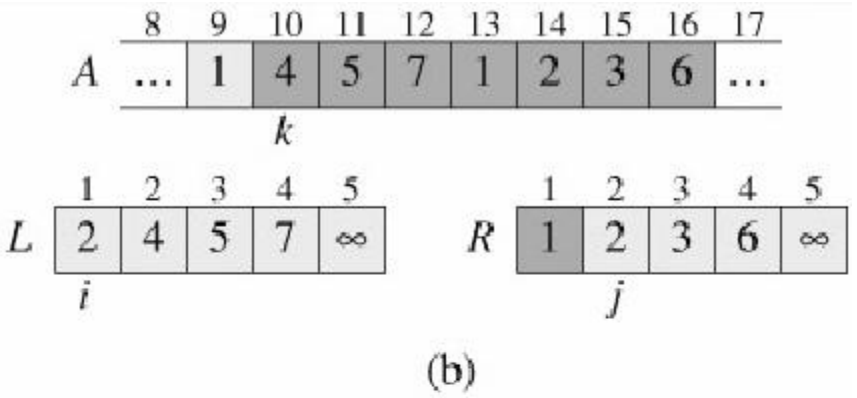
- MERGE伪代码：

```
MERGE(A, p, q, r)
1 n1 ← q - p + 1
2 n2 ← r - q
3 create arrays L[1 .. n1 + 1] and R[1 .. n2 + 1]
4 for i ← 1 to n1
5     do L[i] ← A[p + i - 1]
6 for j ← 1 to n2
7     do R[j] ← A[q + j]
8 L[n1 + 1] ← ∞
9 R[n2 + 1] ← ∞
10 i ← 1
11 j ← 1
12 for k ← p to r
13     do if L[i] ≤ R[j]
14         then A[k] ← L[i]
15              i ← i +1
16         else A[k] ← R[j]
17              j ← j +1
```

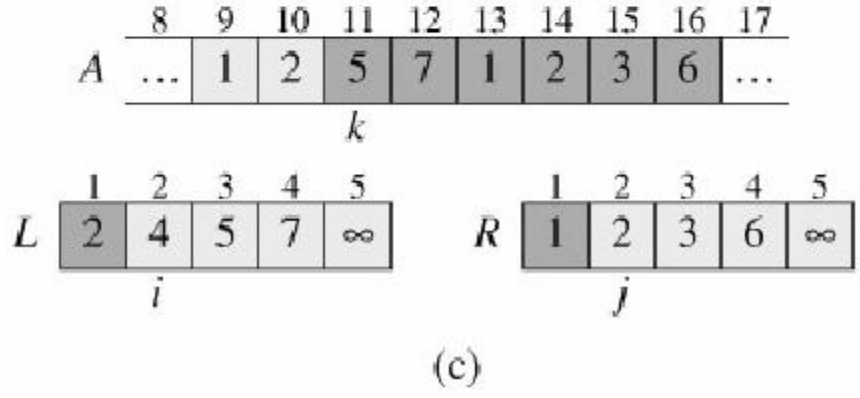
# 2.3.1分治法—Merge过程



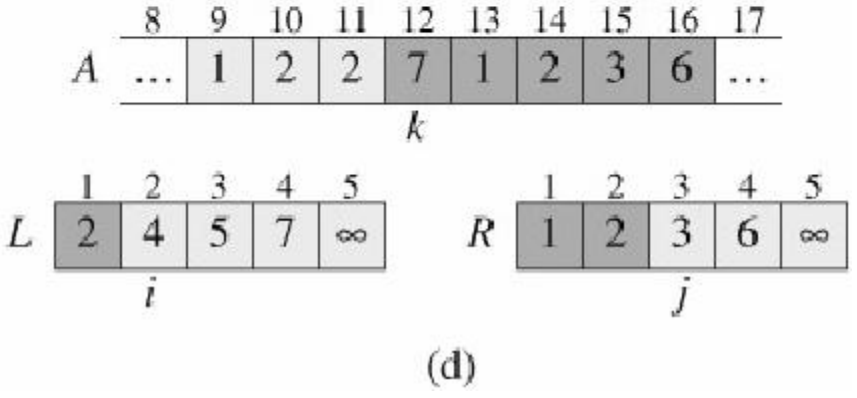

(a)




(b)

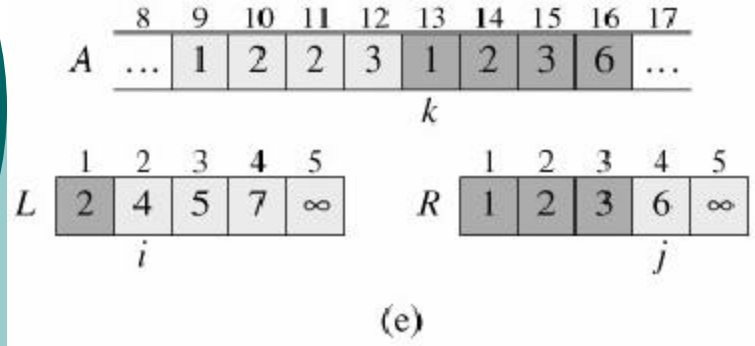



(c)

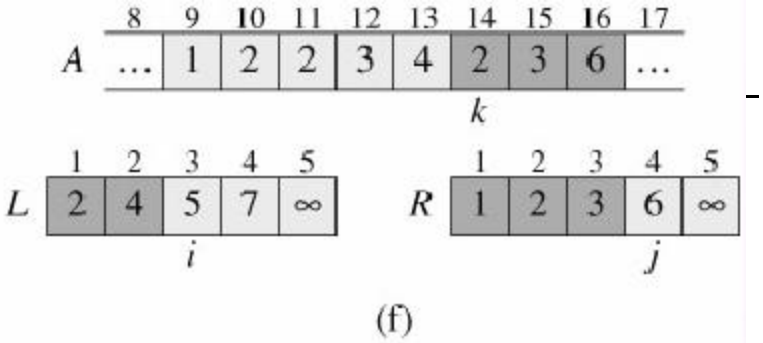



(d)

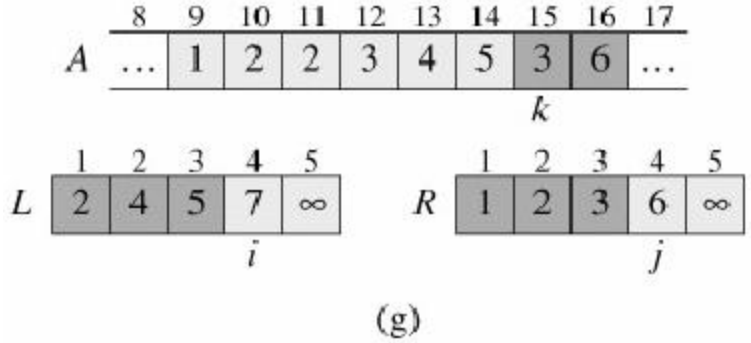
# 2.3.1分治法—Merge过程



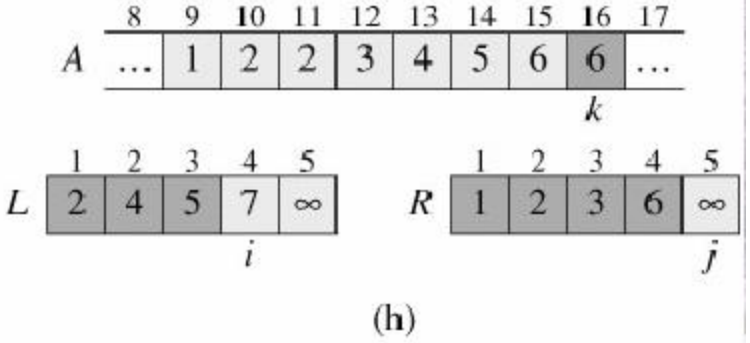

(e)



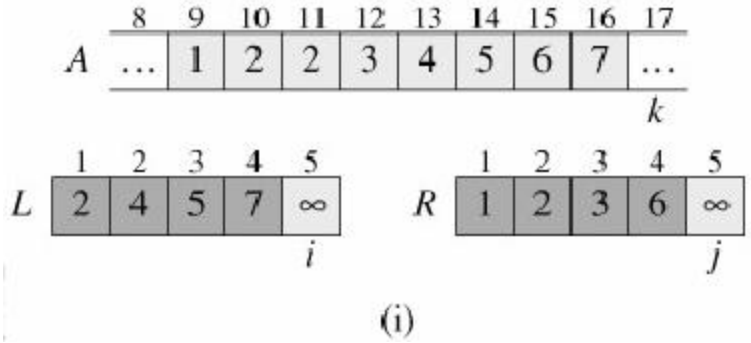

(f)




(g)




(h)




(i)

# 2.3.1分治法

- MERGE过程解释：
  - 第1~2行计算子数组A[p..q]、A[q+1..r]的长度$n_1$ 、$n_2$。
  - 第3行创建了长度$n_1$+1和$n_2$+1数组L和R。
  - 第4~7行中将A[p..q]、A[q+1..r]复制到L、R中去。
  - 第8~9行将哨兵置于数组L和R的末尾。
  - 第10~11行初始化i、j
  - 维护循环不变式：A[p..k-1]包含了输入中排好序的的k-p个最小元素， L[i]和R[j]是各自所在数组中，未被复制回数组A中的最小元素。
  - 第12~17行执行了r-p+1个基本步骤

MERGE伪代码：

```
MERGE(A, p, q, r)
1 n1 ← q - p + 1
2 n2 ← r - q
3 create arrays L[1 .. n1 + 1] and R[1 .. n2 + 1]
4 for i ← 1 to n1
5    do L[i] ← A[p + i - 1]
6 for j ← 1 to n2
7    do R[j] ← A[q + j]
8 L[n1 + 1] ← ∞
9 R[n2 + 1] ← ∞
10 i ← 1
11 j ← 1
12 for k ← p to r
13    do if L[i] ≤ R[j]
14       then A[k] ← L[i]
15            i ← i +1
16       else A[k] ← R[j]
17                    j ← j +1
```

MERGE过程正确性

初始化：在for循环的迭代开始之前，k=p，A[p..k-1]是空的，又i=j=1，L[i]和R[j]是L、R中，尚未被复制回数组A的最小元素。

保持：假设L[i]≤R[j]，L[i]是L、R中未被复制回数组A的最小元素。又A[p..k-1]包含了k-p个最小的元素，因此，第14行将L[i]复制到A[k]中后，子数组A[p..k]将包含k-p+1个最小的元素。

维护循环不变式：
A[p..k-1]包含了输入中排好序的的k-p个最小元素， L[i]和R[j]是各自所在数组中，未被复制
回数组A中的最小元素。

2.3.1分治法

MERGE伪代码：

```
MERGE(A, p, q, r)
1 n1 ← q - p + 1
2 n2 ← r - q
3 create arrays L[1 .. n1 + 1] and R[1 .. n2 + 1]
4 for i ← 1 to n1
5    do L[i] ← A[p + i - 1]
6 for j ← 1 to n2
7    do R[j] ← A[q + j]
8 L[n1 + 1] ← ∞
9 R[n2 + 1] ← ∞
10 i ← 1
11 j ← 1
12 for k ← p to r
13    do if L[i] ≤ R[j]
14       then A[k] ← L[i]
15            i ← i +1
16       else A[k] ← R[j]
17                    j ← j +1
```

MERGE过程正确性(续)

保持(续)：

假设L[i]>R[j]，R[j]是L、R中未被复制回数组A的最小元素。又A[p..k-1]包含了k-p个最小的元素，因此，第16行将R[j]复制到A[k]中后，子数组A[p..k]将包含k-p+1个最小的元素。

终止：在终止时，k=r+1。根据循环不变式，子数组A[p..k-1]为A[p..r]，包含了L和R中k-p=r-p+1个最小元素，并且是已排好序的。

2.3.1分治法

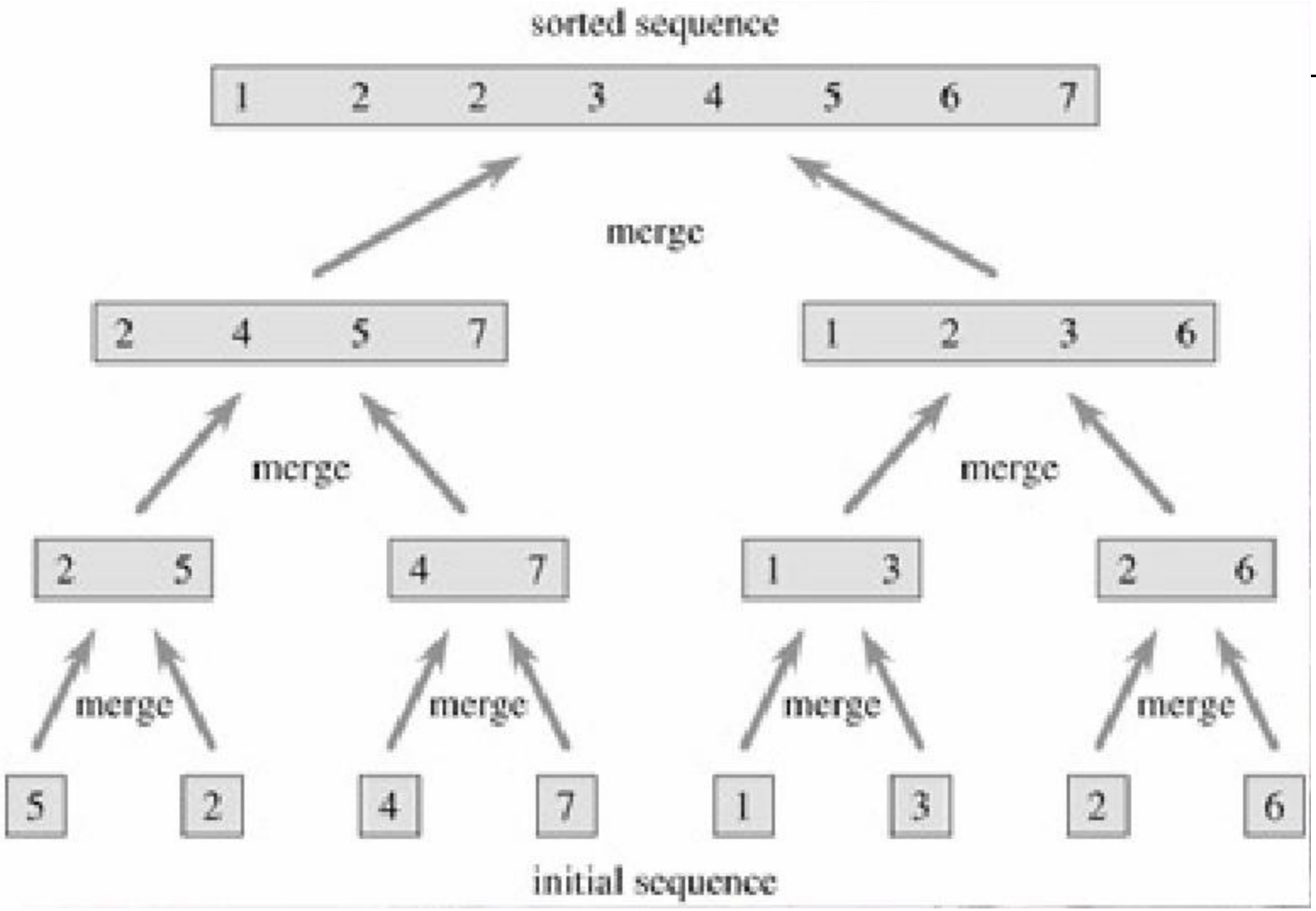
MERGE伪代码：

```
MERGE(A, p, q, r)
1 n1 ← q - p + 1
2 n2 ← r - q
3 create arrays L[1 .. n1 + 1] and R[1 .. n2 + 1]
4 for i ← 1 to n1
5    do L[i] ← A[p + i - 1]
6 for j ← 1 to n2
7    do R[j] ← A[q + j]
8 L[n1 + 1] ← ∞
9 R[n2 + 1] ← ∞
10 i ← 1
11 j ← 1
12 for k ← p to r
13    do if L[i] ≤ R[j]
14       then A[k] ← L[i]
15            i ← i +1
16       else A[k] ← R[j]
17                    j ← j +1
```

Merge过程复杂性(n=r-p+1)：

- 1-3行和8-11行中俄每一行运行时间都是常量
- 4-7行中的for循环所需时间为Θ(n1+n2)=Θ(n)
- 12-17行的for循环共有n轮迭代，其中过每一轮迭代所需时间为常量
- 故：Merge过程的运行时间为Θ(n)

# 2.3.1分治法


sorted sequence
1 2 2 3 4 5 6 7
merge
2 4 5 7
1 2 3 6
merge
merge
2 5
4 7
1 3
2 6
merge
merge
merge
merge
5
2
4
7
1
3
2
6
initial sequence

# 2.3.2分治法分析

- 结构上是递归的
- 运行时间可以用一个递归方程来表示。
  - 设T(n)为一个规模为n的问题的运行时间。
  - 当问题的规模足够的小(如$n \leq c$, c为一个常量)，则得到它的直接解的时间为常量，写作$\Theta(1)$。

- 运行时间可以用一个递归方程来表示。
  - 假设我们把原问题分解成a个子问题，每一个的大小是原问题的1/b。(对于合并排序，a和b都是2，但在许多分治法中a≠b)如果分解该问题和合并解的时间各为D（n）和C（n），则得到递归式：

$$T(n)=\begin{cases}\Theta(1) & \text{如果} n \le x \\ aT(n/b)+D(n)+C(n) & \text{否则}\end{cases}$$

  - 在第4章中，将介绍如何解这类常见的递归式。

# 2.3.2分治法分析

- 合并排序算法的分析
  - 为了简化对基于递归的算法的分析，假定原问题的规模是2的幂次。
  - 分解：仅需计算出子数组中间位置，需要常量时间，因而$D(n)=\Theta(1)$。
  - 解决：递归地解两个规模为n/2的子问题，时间为$2T(n/2)$。
  - 合并：在含有n个元素的子数组上，MERGE过程的运行时间为$\Theta(n)$，则$C(n)=\Theta(n)$。

# 2.3.2分治法分析

- 合并排序算法的分析(续)
  - 合并排序最坏情况运行时间T(n)递归表示：

$$T(n)=\begin{cases}\Theta(1) & \text{如果} n=1\\ 2T(n/2)+\Theta(n) & \text{如果} n>1\end{cases}$$

  - 由“主定理”(第4章将介绍)，T(n)=Θ(nlgn)
  - 对数函数增长速度比任何线性函数都要慢，因此，当输入规模足够大时，合并排序(Θ(nlgn))在最坏情况下比插入排序Θ($n_2$)好。
  - 直观地理解，递归式重写如下：

$$T(n)=\begin{cases}c & \text{如果} n=1\\ 2T(n/2)+cn & \text{如果} n>1\end{cases}$$

## 合并排序算法的分析(续)

- 合并排序最坏情况运行时间T(n)递归表示：

$$\mathrm{T}(n)=\begin{cases} c & \text{如果} n=1 \\ 2T(n/2)+cn & \text{如果} n>1 \end{cases}$$

# Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

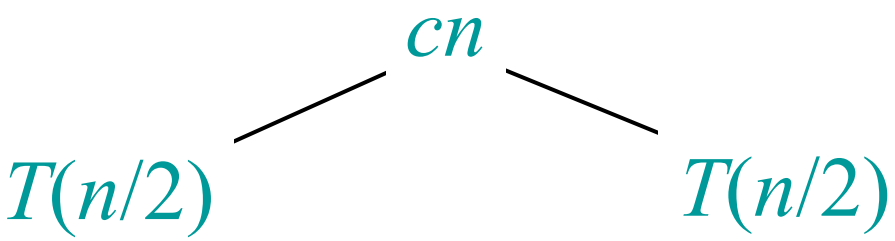
# Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

$T(n)$

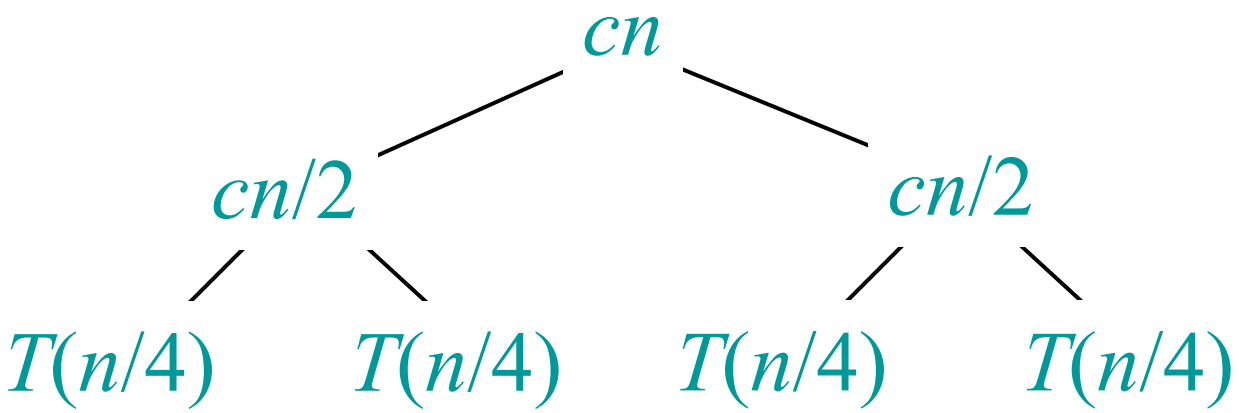
# Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



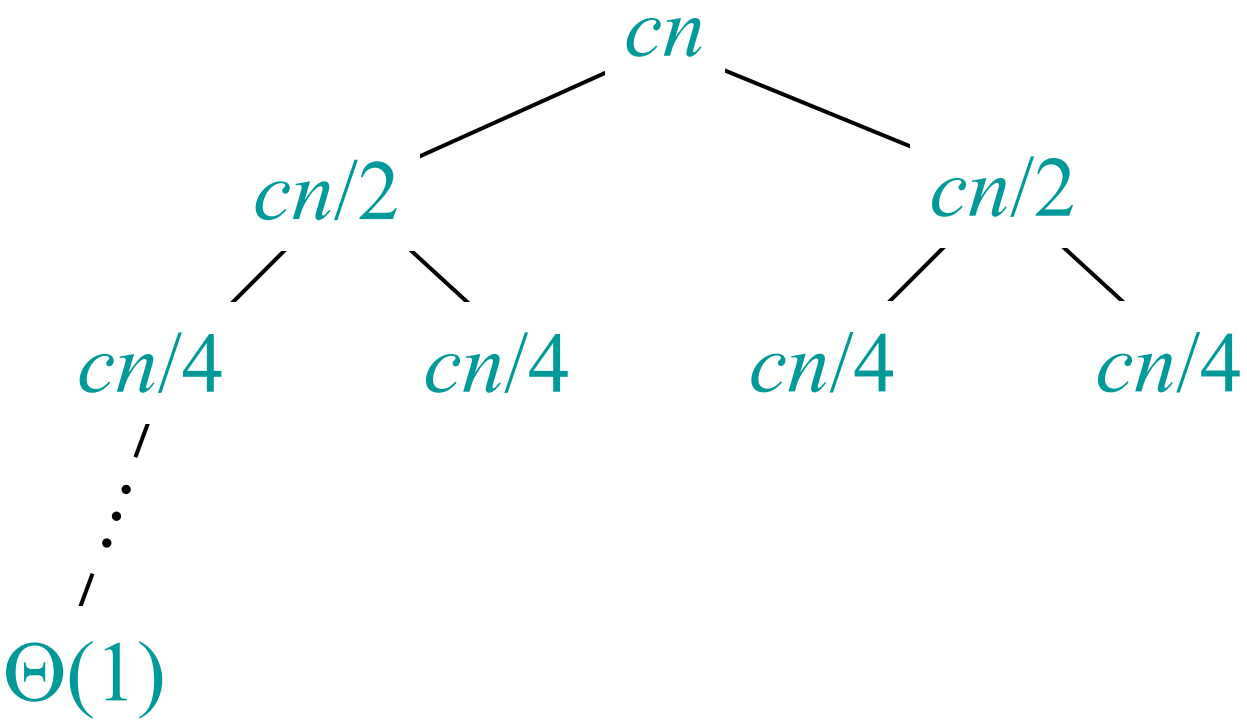
# Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



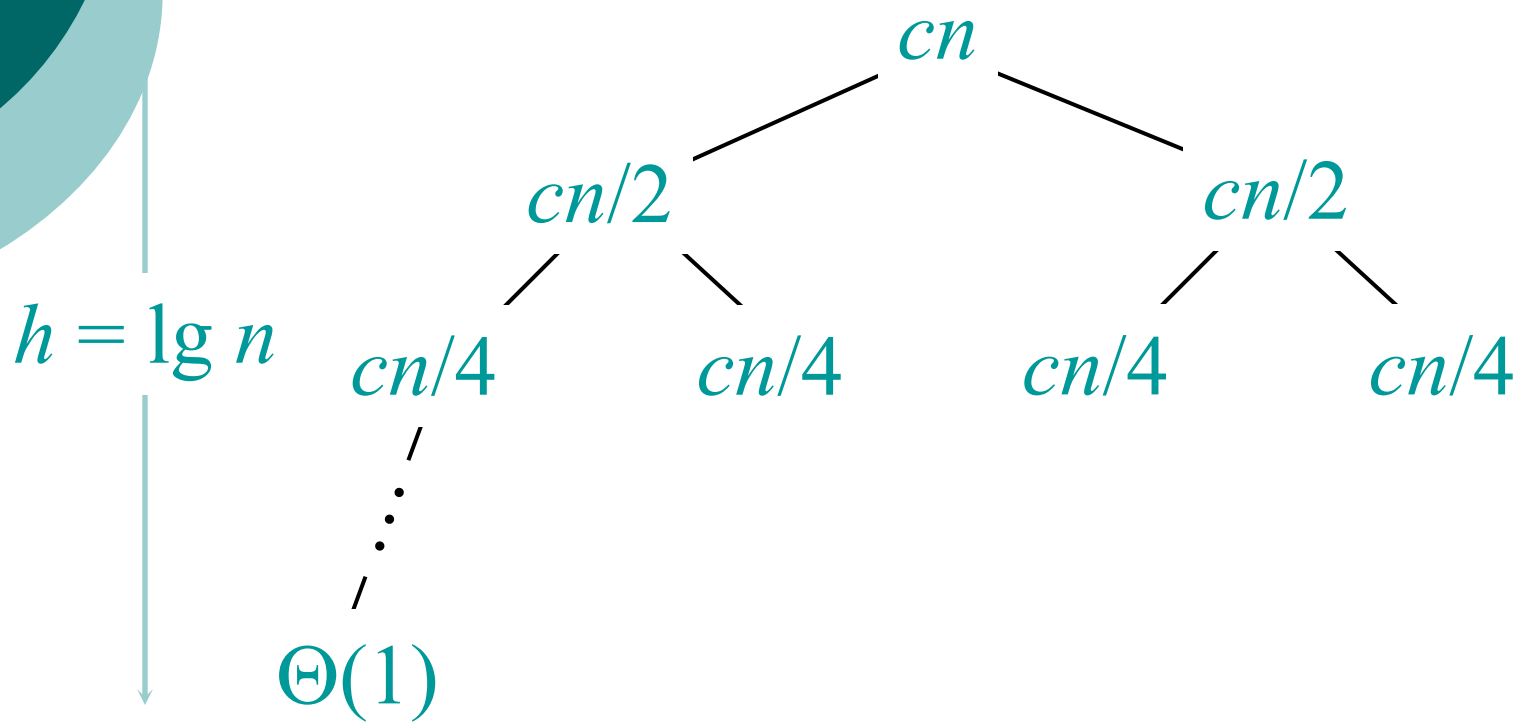
# Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



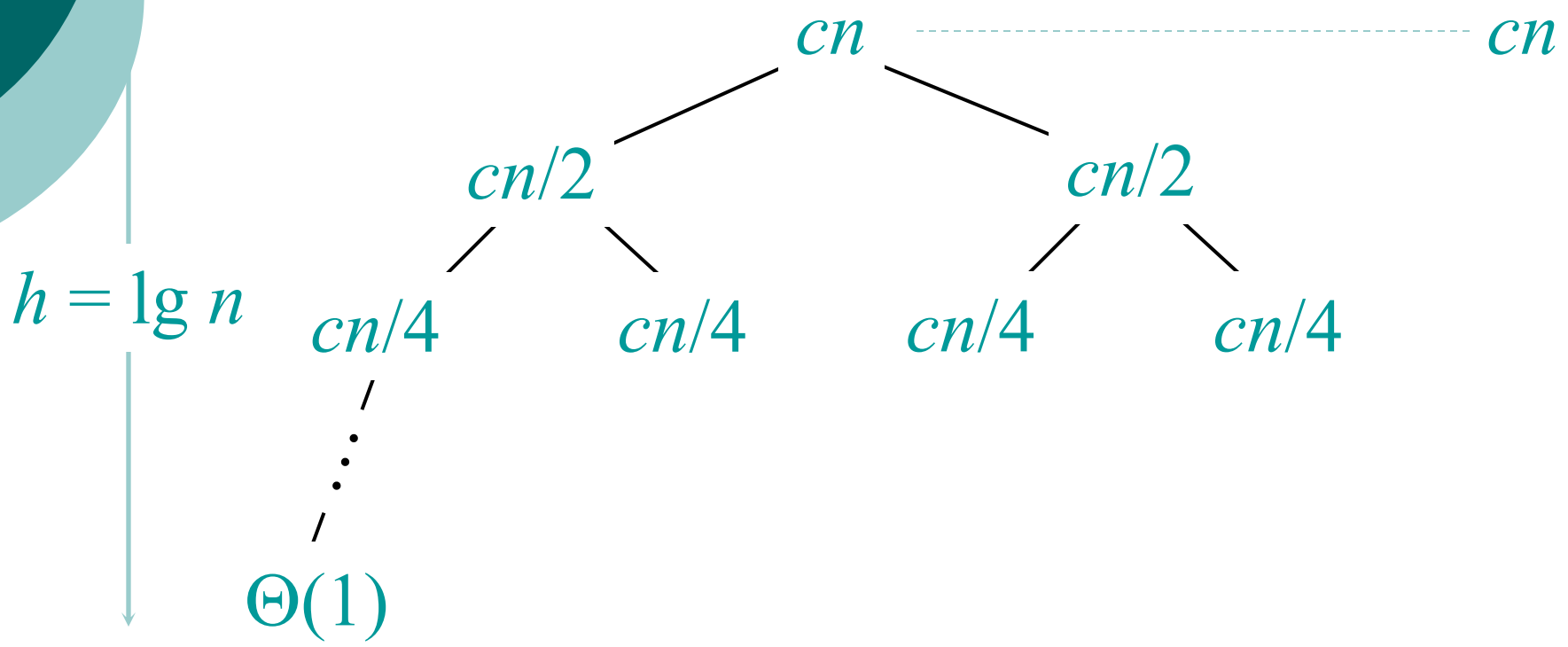
# Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



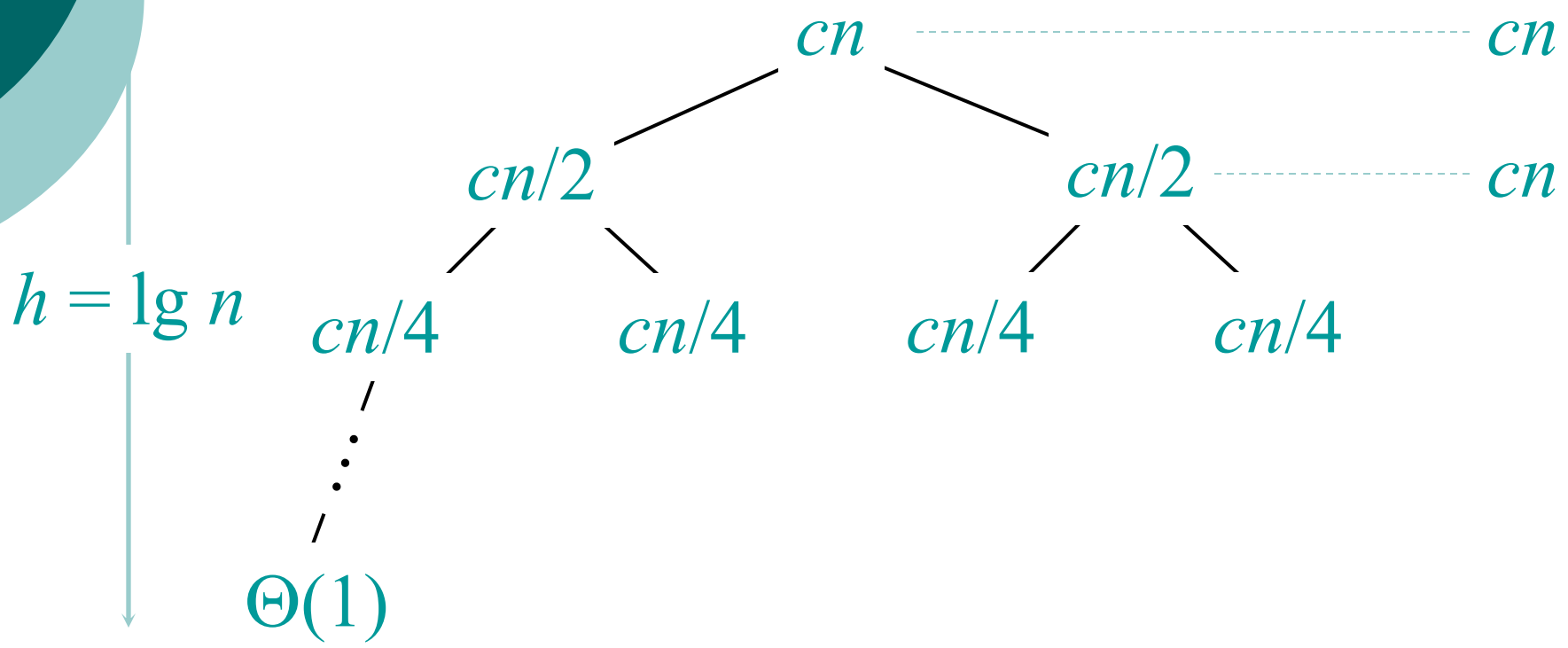
# Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



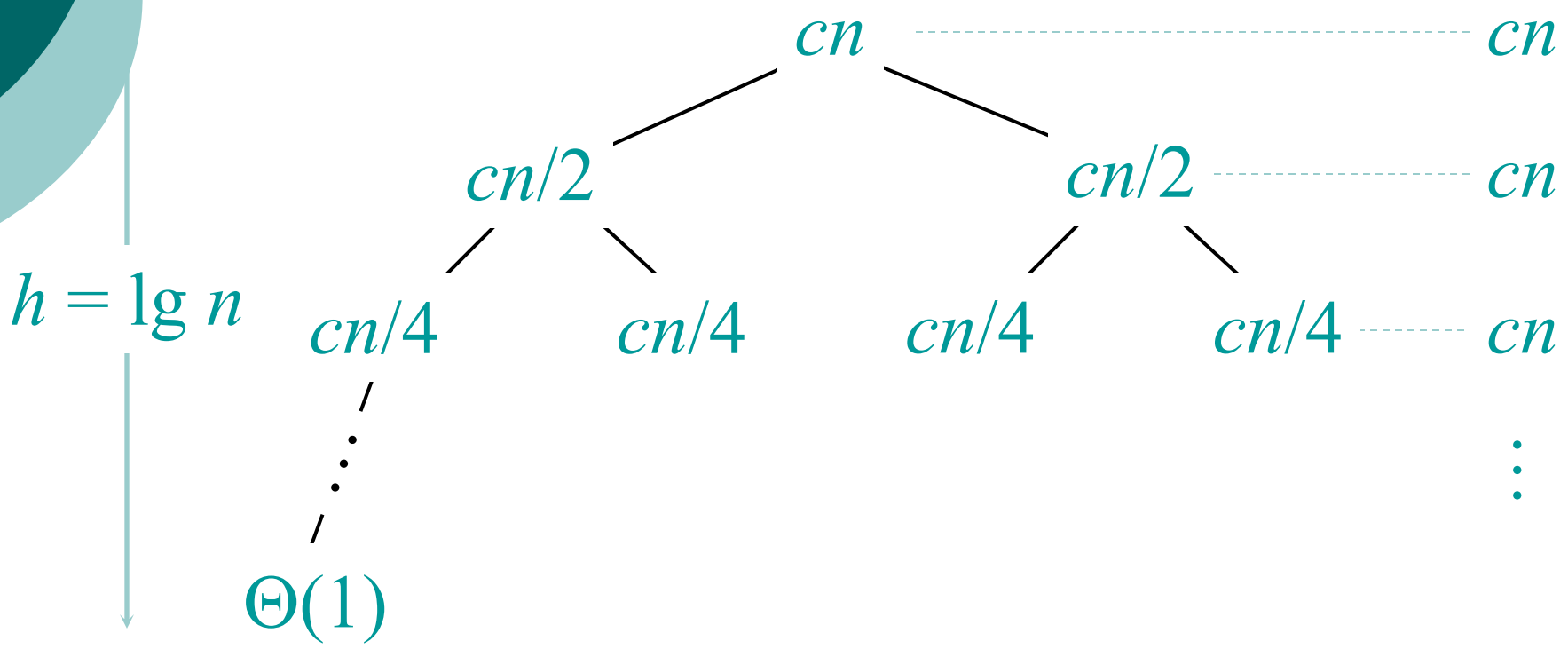
# Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



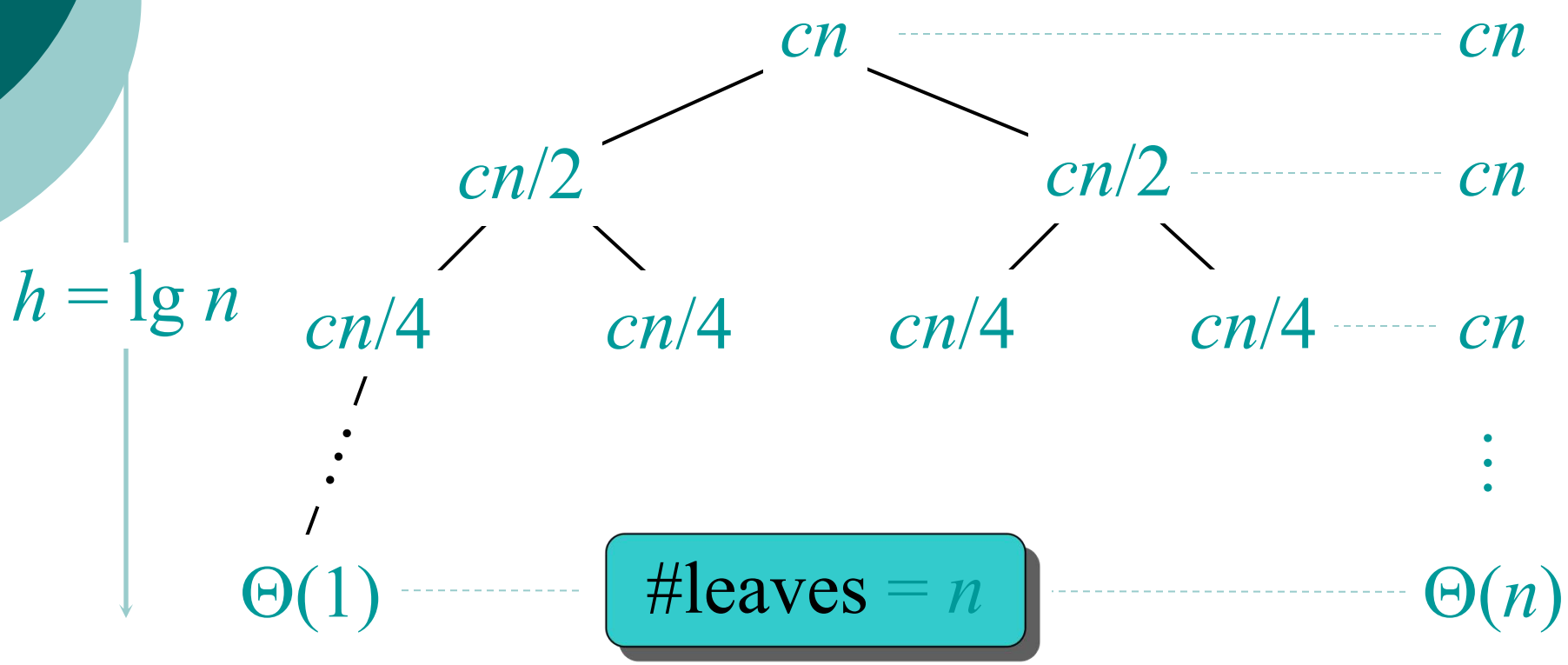
# Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



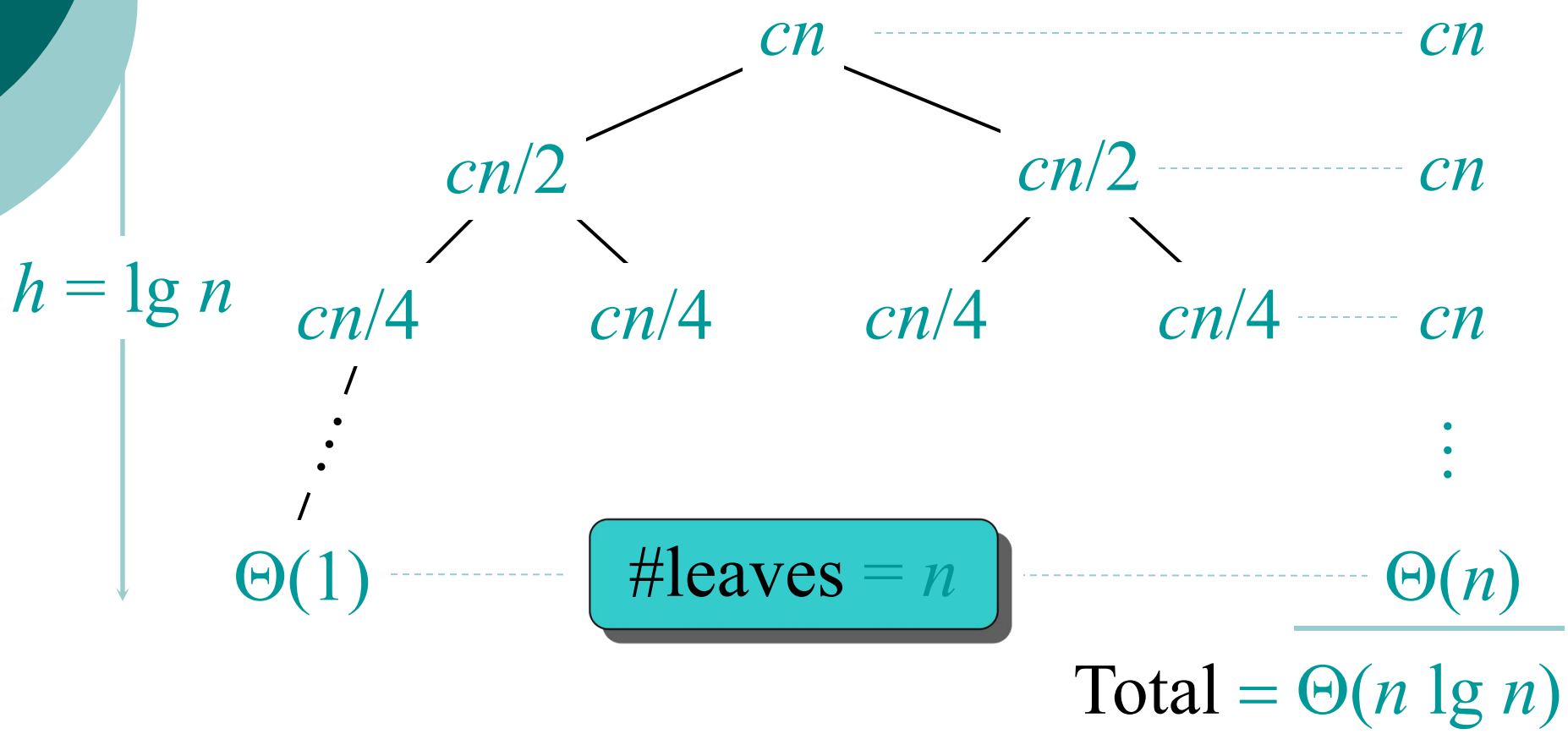
# Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



# Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.




Total = $\Theta(n \lg n)$

# 分治策略小结

## 分治策略的基本思想

**分治策略的实例----二分检索、归并排序**

**主要思想----划分、求解子问题、综合解**

**算法描述**

**Divide-and-Conquer(P)**

1. **if** $|P| \le c$ **then** $S(P)$.
2. **divide** $P$ **into** $P_1, P_2, \ldots, P_k$.
3. **for** $i = 1$ **to** $k$
4. $y_i$ = **Divide-and-Conquer**($P_i$)
5. **Return Merge**($y_1, y_2, \ldots, y_k$)

**注意问题----连续划分　平衡原则**

# 课堂练习2.1

- 分析二分查找算法的分治思想，写出二分查找算法的递归程序描述
- 分析二分查找递归算法的时间复杂度

问题：计算 $a^n$，$n$为自然数

传统算法: $\Theta(n)$.

分治：

$$a^n = \begin{cases} a^{n/2} \times a^{n/2} & n \text{ 为偶数} \\ a^{(n-1)/2} \times a^{(n-1)/2} \times a & n \text{ 为奇数} \end{cases}$$

$$T(n) = T(n/2) + \Theta(1) \Rightarrow T(n) = \Theta(\log n) .$$

# 降低递归算法复杂性的途径

- 位乘问题（乘法算法发展）

1960年，柯尔莫哥洛夫召开算术发展研讨会，他表示：没有一种方法可以以少于 n 的平方次个位数之间的相乘来完成两个n位数之间的相乘。

现场数学家几乎都认同了柯尔莫哥洛夫的观点，一致认为：没有比传统的竖式乘法更好的乘法算法了。

但话音刚落，23岁的俄罗斯数学家Karatsuba（卡拉苏巴）公然表示反对，现场一片哗然。提出知名的Karatsuba算法。

1971年，德国数学家阿诺德·绍哈格(Arnold Schonhage)和沃尔克·斯特拉森(Volker Strassen)的工作取得了真正的突破。他们解释了如何使用快速傅里叶变换(FFT)来有效地对“大数字”做乘法。他们猜测应该有可能使用最多与N log (N) (N乘以N的自然对数)成比例的一些基本运算来乘N位数字。

2019年3月份澳大利亚的两位年轻数学家David Harvey 和Joris van der Hoeven提出新的算法——通过使用多次傅里叶变化，用大量加法和减法代替乘法，证实乘法可以在n×logn步内完成。

# 降低递归算法复杂性的途径

1. 代数变换　减少子问题个数

**例4 位乘问题**

设$X$,$Y$是两个$n$位二进制数，　$n = 2^k$, 求$XY$.

传统算法 $W(n)=O(n^2)$

分治法 令$X= A2^{n/2} +B$, $Y= C2^{n/2} +D$.

$$XY= AC\,2^n + (AD + BC)\,2^{n/2} + BD$$

$$W(n) = 4W(n/2) + cn,$$

$$W(1) = 1$$

解得　$W(n) = O(n^{\log 4}) = O(n^2)$

# 降低递归算法复杂性的途径

代数变换

$$AD + BC = (A - B)(D - C) + AC + BD$$

递推方程

$$W(n) = 3\ W(n/2) + cn$$

$$W(1) = 1$$

解

$$W(n) = O(n^{\log 3}) = O(n^{1.59})$$

# 降低递归算法复杂性的途径

## 例5　Strassen 矩阵乘法

$A,B$ 为两个$n$阶矩阵，$n=2^k$, 计算$C=AB$.

传统算法 $W(n)=O(n^3)$

分治法　将矩阵分块，得

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}\begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}=\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

其中

$$C_{11}=A_{11}B_{11}+A_{12}B_{21} \quad C_{12}=A_{11}B_{12}+A_{12}B_{22}$$

$$C_{21}=A_{21}B_{11}+A_{22}B_{21} \quad C_{22}=A_{21}B_{12}+A_{22}B_{22}$$

递推方程

$$W(n)=8\,W(n/2)+cn^2$$

$$W(1)=1$$

解　$W(n)=O(n^3)$.

# 降低递归算法复杂性的途径

## 分治变换方法

$$M_1 = A_{11}(B_{12} - B_{22}) \qquad M_2 = (A_{11} + A_{12})B_{22}$$

$$M_3 = (A_{21} + A_{22})B_{11} \qquad M_4 = A_{22}(B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_6 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$M_7 = (A_{11} - A_{21})(B_{11} + B_{12})$$

$$C_{11} = M_5 + M_4 - M_2 + M_6 \qquad C_{12} = M_1 + M_2$$

$$C_{21} = M_3 + M_4 \qquad C_{22} = M_5 + M_1 - M_3 - M_7$$

$$W(n) = 7W\left(\frac{n}{2}\right) + 18\left(\frac{n}{2}\right)^2$$

$$W(1) = 1$$

$$W(n) = O(n^{\log_2 7}) = O(n^{2.8075})$$

# 降低递归算法复杂性的途径

## 2. 预处理

递归算法中的处理步尽可能提到递归过程外，作为预处理

**例6　平面点对问题**

输入：集合$S$中有$n$个点，$n>1$，

输出：所有的点对之间的最小距离.

通常算法：$C(n,2)$个点对计算距离，比较最小，需$O(n^2)$时间

分治策略：$O(n\log^2 n)$

# 一维点对问题

输入：集合S中有直线上$n$个点，$n>1$，

输出：所有的点对之间的最小距离

- 直接算法：求出所有点对间的距离后，求最小距离。

  时间：$O(n^2)$

  缺点：复杂度高

- 排序算法：将点对排序，求出相邻点对的距离后求最小

  时间：$O(n \log n)$

  缺点：无法推广

# 一维点对问题（续）

分治算法：

1. 选择分割点$p$ —— 中位数
2. 将$S$分成两部分$S_1$和$S_2$, 分别为$<p$的点和$\geq p$的点.
3. 分别求$S_1$和$S_2$的最小点对$m_1$和$m_2$.
4. 求$S_1$的最大值和$S_2$的最小值，并求它们的差$m_3$.
5. 输出$m_1$、$m_2$、$m_3$的最小值。

时间： $W(n) = 2\,W(n/2) + O(n)$

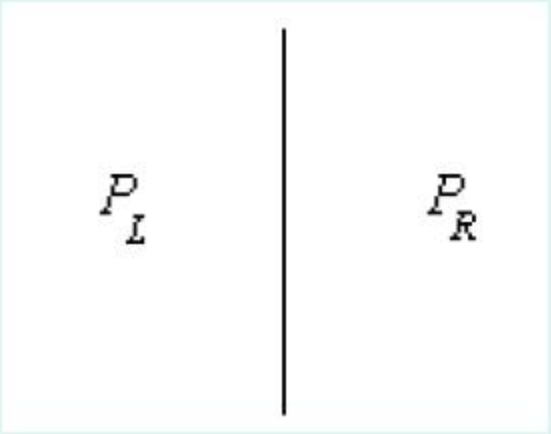
复杂度： $O(n \log n)$

# 平面点对问题的分治算法

分治策略：

取$S$的子集$P$，将$P$中的点划分 成两个子集$P_L$和$P_R$

$$|P_L| = \left\lceil \frac{|P|}{2} \right\rceil \qquad |P_R| = \left\lfloor \frac{|P|}{2} \right\rfloor$$



## 算法2　平面最近点对算法

**MinDistance($P,X,Y$)**

输入：$n$个点的点集$P$,$X$是横坐标的排序数组，$Y$是纵坐标的排序数组

输出：最近的两个点及距离.

1. 如果$P$中点数小于等于3，则直接计算其中的最小距离；
2. 排序$X,Y$;
3. 做垂直线$l$ 将$P$划分为$P_L$和$P_R$，$P_L$的点在$l$左边，$P_R$的点在$l$ 右边
4. MinDidtance($P_L$,$X_L$,$Y_L$); $\delta_L = P_L$中的最小距离；
5. MinDistance($P_R$,$X_R$,$Y_R$); $\delta_R = P_R$中的最小距离；
6. $\delta = \min(\delta_L,\delta_R)$;
7. 对于在垂直线两边距离δ范围内的每个点，检查是否有点与它的距离小于δ，如果存在则将δ修改为新值.

## 算法2　平面最近点对算法

**MinDistance($P,X,Y$)**

输入：$n$个点的点集$P$,$X$是横坐标的排序数组，$Y$是纵坐标的排序数组

输出：最近的两个点及距离.

1. 如果$P$中点数小于等于3，则直接计算其中的最小距离；
2. 排序$X,Y$;
3. 做垂直线$l$ 将$P$划分为$P_L$和$P_R$，$P_L$的点在$l$左边，$P_R$的点在$l$ 右边
4. MinDidtance($P_L,X_L,Y_L$); $\delta_L = P_L$中的最小距离；
5. MinDistance($P_R,X_R,Y_R$); $\delta_R = P_R$中的最小距离；
6. $\delta = \min(\delta_L,\delta_R)$;
7. 对于在垂直线两边距离δ范围内的每个点，检查是否有点与它的距离小于δ，如果存在则将δ修改为新值.



分析：步1　$O(1)$

步2　$O(n\log n)$

步3　$O(1)$

步4-5　$2T(n/2)$

步6　$O(1)$

步7　$O(n)$

$$T(n) = 2T(\frac{n}{2}) + O(n\log n)$$

$$T(n) = O(1) \qquad n \le 3$$

$$T(n) = O(n\log^2 n).$$

# 优化前 算法调用

```
Main()
{
  P(平面顶点集合)
  X（顶点 X 坐标数组）
  Y（顶点 Y 坐标数组）
   MiDistance(P,X,Y)
}
```

**算法2 平面最近点对算法**

**MinDistance(*P*,*X*,*Y*)**

输入：*n*个点的点集*P*,*X*是横坐标的排序数组，*Y*是纵坐标的排序数组

输出：最近的两个点及距离.

1. 如果*P*中点数小于等于3，则直接计算其中的最小距离；
2. 排序*X*,*Y*； O(nlogn)排序算法
3. 做垂直线*l* 将*P*划分为$P_L$和$P_R$，$P_L$的点在*l*左边，$P_R$的点在*l* 右边
4. MinDidtance($P_L$,$X_L$,$Y_L$)；$\delta_L = P_L$中的最小距离；
5. MinDistance($P_R$,$X_R$,$Y_R$)；$\delta_R = P_R$中的最小距离；
6. $\delta = \min(\delta_L,\delta_R)$；
7. 对于在垂直线两边距离δ范围内的每个点，检查是否有点与它的距离小于δ，如果存在则将δ修改为新值.



分析：步1 $O(1)$

步2 $O(n\log n)$

步3 $O(1)$

步4-5 $2T(n/2)$

步6 $O(1)$

步7 $O(n)$

$$T(n) = 2T(\frac{n}{2}) + O(n\log n)$$

$$T(n) = O(1) \qquad n \le 3$$

$$T(n) = O(n\log^2 n).$$

# 预处理优化 算法调用

预处理优化后

```
Main()
{
   P(平面顶点集合)
   X（顶点 X 坐标数组）
   Y（顶点 Y 坐标数组）

   采用 O(nlogn)排序算法对 XY 排序（称为预处理）

    MiDistance(P,X,Y)
}
```

**算法2　平面最近点对算法**

**MinDistance(*P*,*X*,*Y*)**

输入：*n*个点的点集*P*,*X*是横坐标的排序数组，*Y*是纵坐标的排序数组

输出：最近的两个点及距离.

1. 如果*P*中点数小于等于3，则直接计算其中的最小距离；
2. 排序*X*,*Y*；　O(n)算法可以进行处理
3. 做垂直线*l* 将*P*划分为$P_L$和$P_R$，$P_L$的点在*l*左边，$P_R$的点在*l* 右边
4. MinDidtance($P_L$,$X_L$,$Y_L$)；$\delta_L = P_L$中的最小距离；
5. MinDistance($P_R$,$X_R$,$Y_R$)；$\delta_R = P_R$中的最小距离；
6. $\delta = \min(\delta_L, \delta_R)$；
7. 对于在垂直线两边距离δ范围内的每个点，检查是否有点与它的距离小于δ，如果存在则将δ修改为新值.



## 预　处　理

预排序的处理方法

在每次调用时将已经排好的数组分成两个排序的子集，每次调用这个过程的时间为$O(n)$

$$W(n) = T(n) + O(n\log n)$$

$$T(n) = 2T(\tfrac{n}{2}) + O(n)$$

$$T(n) = O(1) \qquad n \le 3$$

解得

$$T(n)=O(n\log n)$$

$$W(n) = O(n\log n)$$

$$T(n) = 2T(\frac{n}{2}) + O(n\log n)$$

$$T(n) = O(1) \qquad n \le 3$$

$$T(n) = O(n\log^2 n).$$

# 课堂练习2.2

- 1、为最近对问题的一维版本设计一个直接基于分治技术的算法。假设输入的点是以升序保存在数组A中。（最近点对问题定义：已知上m个点的集合，找出对接近的一对点。）
- 2、设计一个分治算法来计算二叉树的层数.(空树返回0,单顶点树返回1).
- 3、排列问题：设$R=\{r_1,r_2,...,r_n\}$是 n个不同元素，设计一个递归算法列出所有排列。（可选附加题）